BÖLÜM 2 / MODULE 2

Volatile Anahtar Kelimesi, Atomiklik & Görünürlük

The volatile Keyword, Atomicity & Visibility

Görünürlük (Visibility) ve Atomiklik farkı, `volatile` bellek bariyerleri (Memory Barriers), Double-Checked Locking (DCL) tuzağı, CAS döngüleri ve `LongAdder` vs `AtomicLong` ölçeklenme mimarisi.

Visibility vs Atomicity, hardware memory barriers backing `volatile`, the Double-Checked Locking (DCL) singleton hazard, lock-free CAS loops, and `LongAdder` vs `AtomicLong` scalability.

7

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Görünürlük (Visibility) vs Atomiklik (Atomicity): 'volatile int counter++' Neden Çöker?

Visibility vs Atomicity: Why 'volatile int counter++' is a Silent Catastrophe

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              'volatile int count++' İŞLEMİNİN GERÇEK ANATOMİSİ              │
└──────────────────────────────────────────────────────────┘

 `count++` TEK BİR ATOMİK İŞLEM DEĞİLDİR! 3 AYRI ADIMDAN OLUŞUR:
1. LOAD  : Bellekten o anki değeri oku (Örn: 5)
2. MODIFY: Değere 1 ekle (5 + 1 = 6)
3. STORE : Yeni değeri belleğe yaz (6)

THREAD 1 (Core 0)                    THREAD 2 (Core 1)

┌──────────────────────────┐
│ 1. LOAD: count'u oku (5) │
│                          │
│ 2. MODIFY: 5 + 1 = 6     │
│ 3. STORE: 6'yı belleğe yaz │
│                          │
└──────────────────────────┘
                                  ┌──────────────────────────┐
                                  │                          │
                                  │ 1. LOAD: count'u oku (5) │
                                  │ 2. MODIFY: 5 + 1 = 6     │
                                  │                          │
                                  │ 3. STORE: 6'yı belleğe yaz │
                                  └──────────────────────────┘

BEKLENEN SONUÇ: 7 | GERÇEK SONUÇ: 6! (KAYIP GÜNCELLEME / LOST UPDATE!)
`volatile` SADECE GÖRÜNÜRLÜĞÜ SAĞLAR, ATOMİKLİĞİ (LOCK) SAĞLAMAZ!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki tahtaya tebeşirle sayı yazıldığını düşünün. 'volatile' demek, tahtanın üzerine süper parlak bir spot ışığı yakmak gibidir; tahtada ne yazıyorsa bütün oda anında görür (Görünürlük)! Ama iki kişi aynı anda tahtaya koşup 'Mevcut sayıyı silip 1 fazlasını yazayım' derse ne olur? İkisi de aynı anda '5' sayısını görür, ikisi de kafasında 6 hesaplar, ikisi de tahtaya 6 yazar! 2 kere artması gereken sayı sadece 1 kere artar! Spot ışığı (volatile) insanların birbirinin eline çarpmasını (Atomiklik) engellemez!

🇬🇧 ENGLISH (ELI5)

Imagine a shared blackboard in an office. Declaring a variable `volatile` is like installing a 10,000-lumen spotlight directly onto the board: the instant a number is inscribed, every engineer across the room sees it immediately (Visibility). But what happens when two engineers run to the board at the same second to increment the number? Both observe '5', both mentally calculate '5 + 1 = 6', and both scribble '6' over the board! The count increments once instead of twice! Spotlights guarantee sight, not turn-taking (Atomicity)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Read-Modify-Write Döngüsü ve volatile Sınırları: 1) Görünürlük Garantisi: `volatile` bir değişkene yazıldığında, CPU yerel Store Buffer'ı ana belleğe boşaltır (Flush). Okunduğunda ise CPU yerel cache'i geçersiz kılar (Invalidate) ve RAM/L3'ten taze veri çeker. 2) Atomiklik İhlali: Java bytecode seviyesinde `count++` işlemi: `GETFIELD`, `ICONST_1`, `IADD`, `PUTFIELD` olmak üzere 4 talimata ayrılır. İki thread bu talimatları iç içe (interleaved) çalıştırdığında 'Lost Update' meydana gelir. 3) Kural: `volatile` yalnızca ve yalnızca yazılan yeni değer eski değerden tamamen bağımsızsa (örneğin bir boolean bayrak: `running = false`) güvenlidir. Eğer yeni değer eski değere bağlıysa (`count = count + 1`), `volatile` yetersizdir; `AtomicInteger` veya `synchronized` zorunludur.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Dissecting the Read-Modify-Write Hazard: 1) Hardware Visibility Semantics: A `volatile` write issues a memory barrier draining the local store buffer to the cache coherency fabric. A `volatile` read bypasses stale L1 entries. 2) Bytecode Non-Atomicity: The expression `count++` compiles into non-atomic instructions: `getfield`, `iconst_1`, `iadd`, `putfield`. Context switches occurring between `getfield` and `putfield` produce severe data races (Lost Updates). 3) The Axiom of Volatile: `volatile` is thread-safe IF AND ONLY IF state mutations do not depend on current state (e.g. status flags: `volatile boolean shutdownRequested`). When reads dictate writes, CAS primitives (`AtomicInteger`) or mutual exclusion are non-negotiable.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Volatile Atomiklik İllüzyonu ve Kayıp Güncelleme Kanıtı
public class VolatileBrokenCounter {
    private volatile int count = 0;

    public void increment() {
        count++; // TEHLİKELİ! Atomik değildir!
    }

    void main() throws InterruptedException {
        var demo = new VolatileBrokenCounter();
        int threadCount = 10;
        int incrementsPerThread = 1000;

        Thread[] threads = new Thread[threadCount];
        for (int i = 0; i < threadCount; i++) {
            threads[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < incrementsPerThread; j++) {
                    demo.increment();
                }
            });
        }

        for (Thread t : threads) t.join();

        IO.println("Beklenen Sayaç: " + (threadCount * incrementsPerThread)); // 10,000
        IO.println("Gerçek Sayaç  : " + demo.count); // Çoğunlukla 9,200 civarı çıkar!
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Klasiği: 'Volatile anahtar kelimesi senkronizasyon için ne zaman yeterlidir?' Yanıt: 'Sadece tek bir thread yazıp diğer tüm thread'ler okurken (Single-Writer Multiple-Reader) veya değişken sadece durum bildiren bir bayraksa (`volatile boolean stopped`). Birden fazla thread yazıyorsa volatile tek başına ASLA yetmez!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Top Interview Screen: 'When is volatile sufficient for thread safety?' Senior Answer: 'Exclusively in Single-Writer / Multiple-Reader topologies, or for state flag publication (`volatile boolean cancelled`). If multiple concurrent threads execute read-modify-write operations, volatile guarantees zero safety without locks or CAS operations.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- volatile sadece görünürlüğü (visibility) sağlar; atomiklik (atomicity) sağlamaz.
- count++ arka planda 3 ayrı işlem olduğu için lost update hatasına yol açar.
- Değişkene birden fazla thread yazıyorsa Atomic sınıfları veya kilitler zorunludur.

🎯 Key Takeaways for the Interview

- volatile guarantees memory visibility; it provides zero transactional atomicity.
- count++ encompasses 3 discrete CPU instructions, suffering lost updates under concurrency.
- When multiple threads mutate shared state, Atomic types or mutexes are mandatory.

8

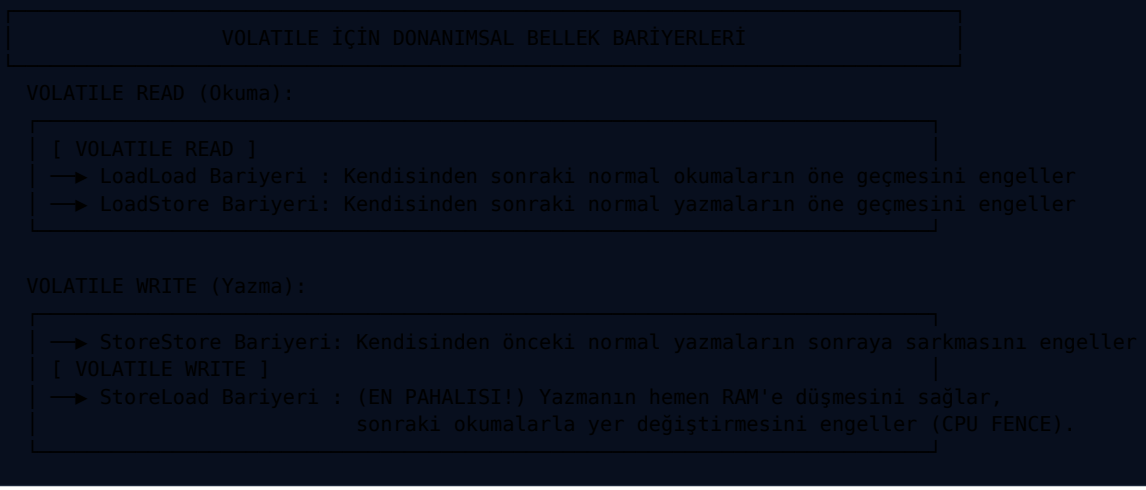
BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

volatile Mekaniği: Donanımsal Bellek Bariyerleri (Memory Barriers / Fences)

The Mechanics of volatile: Hardware Memory Barriers (StoreLoad, StoreStore, LoadLoad)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir postacı düşünün: Mektupları kafasına göre sıraya dizip dağıtıyor (CPU Reordering). Ama patron kapıya bir polis bariyeri (Memory Barrier) dikiyor: 'Bu kırmızı mühürlü mektubu (volatile write) çantandan çıkarmadan önce, elindeki bütün diğer zarfları teslim etmiş olmak zorundasın! Ve bu kırmızı mektubu vermeden sıradaki hiçbir mektubu açamazsın!' İşte Memory Barrier, CPU'nun hızlanmak için yaptığı bütün sıralama hilelerini anında donduran sert bir polis barikatıdır!

🇬🇧 ENGLISH (ELI5)

Think of a courier optimizing deliveries by dropping packages out of order. A Memory Barrier is an armed checkpoint officer standing in the roadway: 'Before you hand over this top-priority VIP document (volatile write), every single regular letter previously in your satchel MUST already be delivered! And you cannot open any future parcels until this VIP document clears!' A Memory Barrier is hardware silicon instructions freezing speculative reordering.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Donanım Seviyesinde 4 Bellek Bariyeri: 1) LoadLoad: `Load1; LoadLoad; Load2` -> Load1 verisi çekilmeden Load2 verisi çekilemez. 2) StoreStore: `Store1; StoreStore; Store2` -> Store1 verisi diğer CPU'lara görünür olmadan Store2 yazılamaz. 3) LoadStore: `Load1; LoadStore; Store2` -> Load1 bitmeden Store2 belleğe basılamaz. 4) StoreLoad (En Ağır Bariyer): `Store1; StoreLoad; Load2` -> Store1'in Store Buffer'dan tamamen temizlenip tüm CPU'lara görünür olmasını garanti eder; aksi takdirde sonraki Load2 bayat veri okuyabilir. x86 mimarisinde donanım zaten güçlü bir bellek modeline (Total Store Order - TSO) sahip olduğu için LoadLoad, LoadStore ve StoreStore bedavadır (NOOP); ancak StoreLoad için pahalı bir `lock addl` veya `mfence` talimatı üretilir. ARM64 ve Apple Silicon zayıf bellek modeline (Weak Memory Model) sahip olduğu için açık `dmb ish` bariyerleri koyar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Hardware Barrier Topologies: 1) LoadLoad & LoadStore: Inhibit subsequent loads/stores from bypassing an initial load. 2) StoreStore: Guarantees preceding stores are flushed to cache hierarchy before subsequent stores become visible. 3) StoreLoad (The Heavyweight Fence): Forces full pipeline serialization. The CPU store buffer must fully drain to L1 before any subsequent load can execute. 4) Architecture Specifics (x86 vs ARM): x86 enforces Total Store Order (TSO) in hardware, making LoadLoad, LoadStore, and StoreStore zero-cost NOOPs; `volatile write` emits a single `lock addl ... 0` instruction to achieve StoreLoad. ARM64/Apple Silicon features a relaxed memory model, emitting explicit `dmb ishld` and `dmb ish` barrier instructions.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Memory Barrier Mantığını Simüle Eden Durum Yayınlama
public class MemoryBarrierPublisher {
    private int configValue = 0;
    private String serverHost = null;
    private volatile boolean initialized = false;

    // Yazan Thread:
    public void initSystem() {
        configValue = 8080;                  // 1. Normal yazma
        serverHost = "api.maang.internal";   // 2. Normal yazma
        // StoreStore Bariyeri buraya yerleşir!
        initialized = true;                  // 3. Volatile yazma (StoreLoad bariyeri tetikler)
    }

    // Okuyan Thread:
    public void handleRequest() {
        if (initialized) { // Volatile okuma (LoadLoad & LoadStore bariyeri)
            // JMM Garantisi: configValue=8080 ve serverHost KESİNLİKLE görünürdür!
            IO.println("Sistem hazır: " + serverHost + ":" + configValue);
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Performans Uyarısı: Mülakatta 'volatile değişken okumak mı yoksa yazmak mı daha pahalıdır?' diye sorulursa; 'Volatile yazma işlemi StoreLoad bariyeri (x86'da mfence / lock prefix) tetiklediği ve CPU Store Buffer'ı boşalmaya zorladığı için yazma işlemi okumadan çok daha pahalıdır!' cevabı verilmelidir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Performance Trap: 'Is a volatile read or a volatile write more expensive?' Senior Answer: 'A volatile write is significantly more expensive on modern CPUs. It forces a StoreLoad memory barrier (e.g. `lock addl` on x86), which stalls execution until the CPU store buffer drains completely to the cache coherency bus.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- volatile anahtar kelimesi CPU seviyesinde Memory Barrier talimatları üretir.
- StoreLoad bariyeri en pahalı bariyerdir; Store Buffer'ı tamamen boşaltır.
- x86 mimarisinde TSO nedeniyle sadece StoreLoad maliyetlidir; ARM64'te açık bariyerler kullanılır.

🎯 Key Takeaways for the Interview

- The volatile keyword compiles into explicit hardware memory barrier instructions.
- The StoreLoad fence is the most expensive barrier, serializing CPU store buffers.
- x86 TSO absorbs most barriers as NOOPs; ARM64 requires explicit fence instructions.

9

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Piggybacking on volatile: Güvenli Durum Yayınlama (Safe Publication)

Piggybacking on volatile: Safe State Publication & Ordering Guarantees

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│              PIGGYBACKING (SIRTA BİNME) HAPPENS-BEFORE KURALI │
└──────────────────────────────────────────────────────────────┘

NON-VOLATILE DEĞİŞKENLER:            VOLATILE DEĞİŞKEN (KORUYUCU MELEK):
int a = 10;
String name = "MAANG";
List<String> items = new ArrayList();
items.add("Data");
                                     volatile int trigger = 1; // ◄── YAZMA!
──────────────────────────────────────────────────────────────
BÜTÜN BU NORMAL DEĞİŞKENLER, TRIGGER'IN ARKASINA TAKILARAK RAM'E BASILIR!

OKUYUCU THREAD:
if (trigger == 1) { // ◄── VOLATILE OKUMA!
    // a, name ve items listesi KESİNLİKLE tam ve güncel olarak görünür!
}
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Havaalanında uçağa binerken pasaportu olmayan 3 çocuk düşünün (Normal değişkenler). Kapıdaki polis tek tek çocukları içeri almaz. Ama yanlarında diplomatik pasaporta sahip bir büyükelçi (volatile değişken) vardır. Büyükelçi kapıdan geçerken 'Bu çocuklar benimle' dediği anda, bütün çocuklar büyükelçinin sırtına binerek (Piggybacking) içeriye güvenle sızarlar! Tek bir volatile değişken, kendisinden önce yazılan onlarca normal değişkeni de beraberinde görünür kılar!

🇬🇧 ENGLISH (ELI5)

Imagine three passengers lacking boarding passes (regular non-volatile fields). Airport security refuses them entry. However, they walk alongside an accredited diplomat holding an all-access pass (a single `volatile` variable). When the diplomat clears the gate, the Happens-Before protocol dictates that everyone preceding the diplomat in line is admitted simultaneously (Piggybacking). A single volatile write flushes all preceding non-volatile state mutations into global visibility!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Piggybacking Mekanizması ve JMM Transitivity Kuralı: 1) Happens-Before Transitivity: Eğer `A -> B` (Program Order) ve `B -> C` (Volatile kuralı) ise, o zaman `A -> C`'dir. Burada A normal değişken yazmaları, B volatile yazma, C ise başka bir thread'in o volatile değişkeni okumasıdır. 2) Java Standart Kütüphanesindeki Yeri: `java.util.concurrent` paketindeki birçok sınıf (ör. `AbstractQueuedSynchronizer` - AQS ve `ConcurrentHashMap`) bu hileyi yoğun şekilde kullanır. Tüm alanları `volatile` yapmak yerine, tek bir `volatile int state` değişkenine yazarak sınıftaki diğer tüm düz alanları görünür kılarlar. Bu sayede gereksiz bellek bariyeri maliyetlerinden kaçınarak inanılmaz bir throughput elde edilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Piggybacking Formalism & JDK Engineering: 1) Transitively Inherited Visibility: JSR-133 guarantees that if Action A (non-volatile writes) happens-before Action B (volatile write), and Action B happens-before Action C (volatile read by Thread 2), then Action A happens-before Action C. 2) High-Performance JDK Exploitation: `AbstractQueuedSynchronizer` (AQS) powers ReentrantLock, Semaphore, and CountDownLatch. AQS does not mark internal queue node pointers as volatile; instead, it relies on piggybacking off mutations to `volatile int state` to safely publish entire linked-node updates, evading catastrophic memory fence penalties.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Piggybacking ile Düz Değişkenleri Güvenle Yayınlama
import java.util.HashMap;
import java.util.Map;

public class PiggybackingPublisher {
    // Bu map normal bir HashMap'tir (Thread-safe DEĞİLDİR!)
    private Map<String, String> internalConfig = new HashMap<>();

    // Sihirli koruyucu: Tek bir volatile tetikleyici
    private volatile boolean published = false;

    public void prepareAndPublish() {
        // 1. Normal, senkronize olmayan map manipülasyonu
        internalConfig.put("db.user", "admin");
        internalConfig.put("db.password", "secret");
        internalConfig.put("cache.ttl", "3600");

        // 2. Piggybacking tetikleyici: Volatile yazma!
        // Kendisinden önceki BÜTÜN map verilerini bellek bariyeriyle RAM'e döker.
        published = true;
    }

    public void readConfig() {
        if (published) { // Volatile okuma
            // internalConfig KESİNLİKLE eksiksiz ve güvenle okunur!
            IO.println("Kullanıcı: " + internalConfig.get("db.user"));
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Kritik Kodlama Tuzağı: Piggybacking yaparken kod sıralamasına çok dikkat edilmelidir! Eğer `published = true;` satırını map'e veri eklemeden ÖNCE yazarsanız, reordering yüzünden okuyucu thread boş veya yarım doldurulmuş map ile karşılaşır!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Dangerous Trap: The order of statements is lethal in piggybacking! If you assign `published = true` BEFORE mutating `internalConfig`, the happens-before chain is severed. Observers will read partially constructed, corrupted data structures.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Piggybacking, normal değişkenleri volatile bir yazmanın arkasına takarak görünür kılmaktır.
- JMM'nin Geçişlilik (Transitivity) kuralı sayesinde A -> B ve B -> C ise A -> C olur.
- JDK'nın AQS altyapısı yüksek performans için piggybacking prensibini kullanır.

🎯 Key Takeaways for the Interview

- Piggybacking piggybacks non-volatile updates onto the back of a volatile write.
- Happens-before transitivity legally guarantees that preceding non-volatile state is visible.
- JDK concurrency engines (AQS) leverage piggybacking to minimize memory fence overhead.

10

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Double-Checked Locking (DCL) & Singleton Tuzağı: volatile Neden Zorunlu?

Double-Checked Locking (DCL) & The Singleton Trap: Why volatile is Non-Negotiable

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              DOUBLE-CHECKED LOCKING (DCL) NESNE OLUŞTURMA ADIMLARI              │
└──────────────────────────────────────────────────────────┘

`instance = new Singleton();` İŞLEMİ TEK BİR ADIM DEĞİLDİR!

GERÇEK KOD ADIMLARI:
1. memory = allocate();   // Bellekte yer tahsis et
2. ctorSingleton(memory); // Constructor'ı çalıştır (Alanları doldur)
3. instance = memory;     // Referansı instance değişkenine bağla

CPU / JIT YENİDEN SIRALAMASI (REORDERING):
1. memory = allocate();   // Bellek tahsis edildi
3. instance = memory;     // ⚠ DİKKAT! Referans erkenden bağlandı! (İçi henüz BOŞ!)
2. ctorSingleton(memory); // Constructor hala çalışıyor...

THREAD 2 TAM BU ANDA GELİR:
if (instance == null) ──► FALSE DÖNER! (Çünkü 3. adım çalıştı!)
return instance;      ──► 💣 GÜM! Thread 2 henüz İÇİ BOŞ nesneyi kullanmaya başlar!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir mobilyacıdan gardırop siparişi verdiniz. Usta dükkanda parçaları birleştirip öyle teslim etmelidir. Ama usta hızlanmak için boş gardırop kolisini (Bellek tahsisi) evinize getirip kapıya koyar (Referansı bağlar), raflarını ve kapaklarını ise sonradan takmaya başlar. Tam o sırada evin çocuğu gelip kapağı olmayan, vidaları takılmamış gardırobun içine kıyafet koymaya çalışırsa gardırop çocuğun üstüne devrilir! İşte 'volatile' koymazsanız, Java nesnenin bitmemiş kolisini diğer thread'lere dolu gibi gösterir!

🇬🇧 ENGLISH (ELI5)

You order an assembled oak wardrobe. The carpenter should assemble the doors and drawers before handing over the keys. Instead, the rushed carpenter sets a hollow wooden frame in your bedroom (instance pointer published) while rummaging for screws and shelves in the hallway (constructor executing). If your child immediately tosses coats inside, the unfinished frame collapses! Without `volatile`, the JVM publishes the object reference BEFORE constructor completion, causing threads to access half-baked objects!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

DCL Probleminin Çözümü ve `volatile`: 1) Neden synchronized yetersiz?: Metodun tamamını `synchronized` yapmak her okumada kilit maliyeti yaratır. Bu yüzden 'önce kilitsiz kontrol et, null ise kilide gir, içeride bir daha kontrol et' (Double-Check) tasarlandı. 2) Yarı Yapılandırılmış Nesne Kaçağı (Partially Constructed Object): JIT derleyicisi 2. ve 3. adımların yerini değiştirebilir (Instruction reordering). Thread A henüz constructor'ı bitirmeden `instance` referansı non-null hale gelir. O anda kilit dışındaki Thread B ilk kontrolde `instance != null` görür ve nesneyi alır; ancak nesnenin alanları henüz varsayılan `0` veya `null` değerindedir! 3) volatile Çözümü: `private static volatile Singleton instance;` tanımlandığında, StoreStore ve StoreLoad bariyerleri sayesinde nesnenin constructor'ı tamamen bitmeden referans ASLA yayınlanamaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Deconstructing Double-Checked Locking: 1) The Performance Impetus: Synchronizing the entire `getInstance()` method incurs mutex acquisition on every single read. DCL seeks to synchronize strictly on the initial null check. 2) Reordering Vulnerability: The JVM allows reordering between step 2 (instance initialization) and step 3 (pointer assignment). Thread A executes step 1 then step 3; Thread B executes the outer unsynchronized check, observes `instance != null`, and happily returns the instance while Thread A is still executing step 2! Thread B observes a partially constructed object, crashing with NPEs or corrupt data. 3) The Volatile Fix: Declaring `volatile Singleton instance` forces a StoreStore barrier prior to reference assignment, ensuring all constructor writes flush before publication.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Güvenli Double-Checked Locking (DCL) Singleton
public class SafeDclSingleton {
    // ⚠ KRİTİK: volatile OLMADAN DCL %100 HATALIDIR!
    private static volatile SafeDclSingleton instance;
    private final String connectionString;

    private SafeDclSingleton() {
        // Ağır başlatma simülasyonu
        this.connectionString = "jdbc:postgresql://db.maang.internal:5432/core";
    }

    public static SafeDclSingleton getInstance() {
        // 1. Kontrol (Kilitsiz - Hızlı Okuma)
        SafeDclSingleton localRef = instance;
        if (localRef == null) {
            synchronized (SafeDclSingleton.class) {
                // 2. Kontrol (Kilit İçinde - Kesin Tekillik)
                localRef = instance;
                if (localRef == null) {
                    instance = localRef = new SafeDclSingleton();
                }
            }
        }
        return localRef;
    }

    public String getConnectionString() {
        return connectionString;
    }

    void main() {
        var s = SafeDclSingleton.getInstance();
        IO.println("Bağlantı Başarılı: " + s.getConnectionString());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakatın 1 Numaralı Tuzağı: Mülakatçı size DCL kodunu tahtaya yazar ve 'Burada volatile'ı unutsak ne olur?' der. 'NullPointerException veya beklenmedik değerler alırız çünkü başka bir thread henüz constructor'ı tamamlanmamış yarı-yapılandırılmış (partially constructed) bir nesneye erişir' cevabını vermeyen aday elenir!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

The 1 MAANG Concurrency Question: 'What breaks if we omit volatile in Double-Checked Locking?' Answer: 'Threads will observe a partially constructed object! The compiler or CPU can reorder memory assignment before constructor invocation, exposing uninitialized fields to unsynchronized reader threads.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- DCL deseninde volatile kullanımı bir seçenek değil, matematiksel bir zorunluluktur.
- volatile olmadan başka bir thread yarı-yapılandırılmış (partially constructed) nesne görür.
- Modern Java'da alternatif olarak 'Initialization-on-demand holder' idiom tercih edilebilir.

🎯 Key Takeaways for the Interview

- volatile in Double-Checked Locking is a strict requirement, not an optional optimization.
- Without volatile, reader threads risk observing uninitialized, partially constructed state.
- Alternatively, use the Bill Pugh Initialization-on-demand holder idiom.

11

BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Atomik İlkel Tipler: AtomicInteger, CAS (Compare-And-Swap) & Kilit-Siz Programlama

Atomic Primitives: AtomicInteger, CAS (Compare-And-Swap) & Lock-Free Computing

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────┐
│              COMPARE-AND-SWAP (CAS) DONANIMSAL DÖNGÜSÜ   │
└──────────────────────────────────────────────────────────┘

Thread 1 ve Thread 2 aynı anda count'u (5) artırmak istiyor:

1. Thread 1 okur: Beklenen Değer = 5, Yeni Değer = 6
2. Thread 2 okur: Beklenen Değer = 5, Yeni Değer = 6

CPU DONANIM SEVİYESİNDE ATOMİK TALİMAT: `CMPXCHG` (Compare and Exchange)

Thread 1 CPU'ya sorar: "Bellekteki değer hala 5 mi?"
CPU: "Evet, hala 5. Değeri 6 yapıyorum!" ──► [ BAŞARILI (CAS SUCCESS) ]

Thread 2 CPU'ya sorar: "Bellekteki değer hala 5 mi?"
CPU: "HAYIR! Değer artık 6 oldu!"        ──► [ BAŞARISIZ (CAS FAIL) ]

Thread 2 pes etmez (Spin / Retry Loop):
Yeniden okur: Beklenen = 6, Yeni = 7 ──► Tekrar dener ──► BAŞARILI!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir banko düşünün. Kilitli sistemde (synchronized) veznedar içeri tek bir müşteri alır, kapıyı kilitler, işi bitene kadar dışarıdaki 100 kişi yağmurda bekler. Kilitsiz sistemde (CAS / Atomic) ise kapı her zaman açıktır! Herkes vezneye koşar: 'Hesabımda 100 TL olduğunu biliyorum, onu 150 yap!' der. Veznedar bakar: 'Evet gerçekten 100 TL var, 150 yaptım (Başarılı).' İkinci kişi geldiğinde 'Hesabımda 100 TL vardı 120 yap' derse veznedar: 'Geç kaldın, para artık 150 oldu! Git sıranı tekrar hesapla gel!' der. Müşteri dışarıda beklemez, hemen güncel paraya göre tekrar dener!

🇬🇧 ENGLISH (ELI5)

Consider an amusement park turnstile. Pessimistic locking (synchronized) puts an armed guard at the gate: only one person enters, while 500 people wait frozen in the cold. Optimistic lock-free computing (CAS) removes the guard entirely! Everyone pushes forward: 'I observed the ticket counter was at 100; advance it to 101!' The hardware checks: 'Is it still 100? Yes! Flipping to 101 (Success).' If another person arrives claiming it was 100, hardware rejects: 'False! It is already 101! Recalculate and retry.' Nobody sleeps; losers spin and retry instantly!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CAS Mekanizması ve Donanım Desteği: 1) `CMPXCHG` Talimatı: x86 mimarisinde `lock cmpxchg` tek bir CPU saat çevriminde atomik olarak belleği kontrol eder ve eşitse yazar. Bu işlem işletim sistemi seviyesinde thread askıya alma (park/unpark) gerektirmez; kullanıcı modunda (user space) ultra hızlı gerçekleşir. 2) İyimser Kilitlenme (Optimistic Concurrency): Çatışmanın düşük olduğu sistemlerde CAS, geleneksel mutex kilitlerden 10 kat daha hızlıdır. 3) Spin-Wait Maliyeti: Yüksek çekişmede (high contention), onlarca thread aynı anda CAS denerse CPU döngüleri boşa harcanır (CPU spike) ve throughput düşer.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Hardware CAS Mechanics: 1) `cmpxchg` Assembly Primitive: AtomicInteger relies on `Unsafe` / `VarHandle.compareAndSet()`, executing the hardware-level `lock cmpxchg` instruction. It evaluates `if (memory == expected) { memory = update; return true; } else return false;` in a single un-interruptible silicon cycle. 2) Lock-Free Progress Guarantee: At least one thread is mathematically guaranteed to make forward progress on every CAS collision round, preventing priority inversions and deadlock. 3) Contention Penalty: Under extreme multi-threaded writes, CAS failure rates skyrocket, generating spin-wait CPU burn as threads continuously thrash on cache-line invalidations.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: AtomicInteger ile Thread-Safe Lock-Free Sayaç
import java.util.concurrent.atomic.AtomicInteger;

public class LockFreeCounter {
    private final AtomicInteger count = new AtomicInteger(0);

    public void increment() {
        // Dahili olarak compareAndSet döngüsü (CAS loop) işletir
        count.incrementAndGet();
    }

    public int get() {
        return count.get();
    }

    // Manuel CAS Döngüsü Örneği (Koşullu Güncelleme)
    public void multiplyIfLessThan(int multiplier, int limit) {
        int current;
        int next;
        do {
            current = count.get();
            if (current >= limit) return;
            next = current * multiplier;
        } while (!count.compareAndSet(current, next)); // Eşitse güncelle, değilse tekrar dene!
    }

    void main() {
        var counter = new LockFreeCounter();
        counter.increment();
        counter.multiplyIfLessThan(5, 10);
        IO.println("Lock-Free Son Değer: " + counter.get());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'AtomicInteger kullanırsak thread'ler hiçbir zaman bloke olmaz mı?' Yanıt: 'Evet, işletim sistemi seviyesinde BLOCKED olmazlar; ANCAK yüksek çekişmede (High Contention) CAS sürekli başarısız olup döngüye (spin) gireceği için CPU kullanımı %100'e fırlayabilir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Trap: 'Does AtomicInteger eliminate all thread blocking overhead?' Answer: 'Threads never enter the OS BLOCKED state; however, under extreme contention, repeated CAS failures cause aggressive spinning, spiking CPU utilization to 100% while starving other tasks.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- CAS (Compare-And-Swap), CPU donanım seviyesinde atomik kontrol ve yazma yapar.
- Kilit kullanılmadığı için Deadlock riski sıfırdır.
- Aşırı çekişmede CAS döngüleri CPU tüketimini tavan yaptırabilir.

🎯 Key Takeaways for the Interview

- CAS (Compare-And-Swap) executes atomic evaluate-and-set in hardware silicon via CMPXCHG.
- Lock-free algorithms carry zero risk of mutual deadlocks.
- Severe contention causes CAS spin-loops to burn excessive CPU cycles.

12

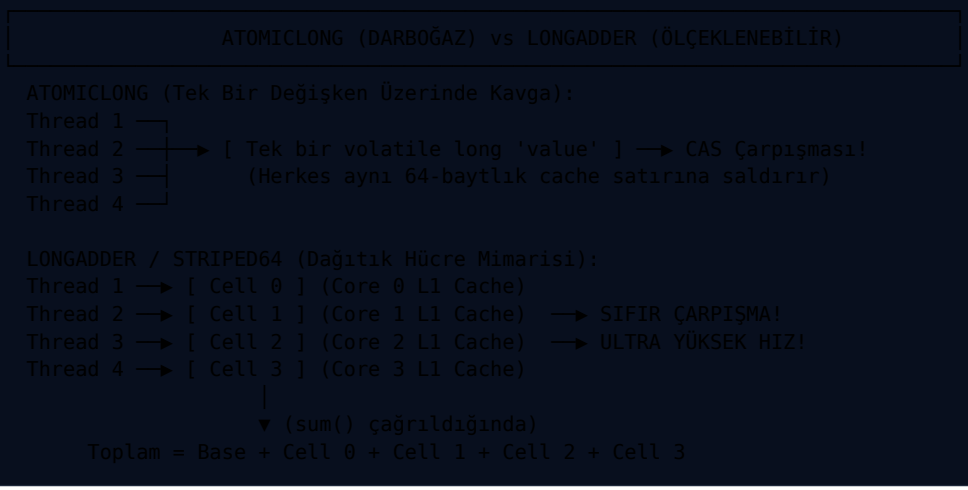
BÖLÜM: VOLATILE ANAHTAR KELIMESI, ATOMIKLIK & GÖRÜNÜRLÜK | MODULE: THE VOLATILE KEYWORD, ATOMICITY & VISIBILITY

Yüksek Ölçekli Sayaçlar: LongAdder vs AtomicLong (Striped64 & Dağıtık Hücreler)

High-Throughput Scalability: LongAdder vs AtomicLong (Striped64 & Cell Arrays)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir süpermarkette tek bir kasa olduğunu düşünün (AtomicLong). 50 müşteri aynı anda tek bir kasiyere para ödemeye çalışırsa birbirini ezer, kuyruk uzar. LongAdder ise süpermarkete 8 ayrı kasa açmak gibidir! Müşteriler farklı kasalara (Cell) dağılır, kimse kimseyi beklemeden parasını öder. Günün sonunda patron kasaların hepsini toplar (sum()) ve toplam ciroyu bulur! Okuma nadir, yazma çoksa LongAdder 100 kat daha hızlıdır!

🇬🇧 ENGLISH (ELI5)

Picture a stadium with a single concession register (AtomicLong). Thousands of fans violently rush that lone cashier simultaneously, resulting in massive bottlenecks and dropped orders. LongAdder opens 16 parallel registers distributed across the stadium (Cell array). Fans transact at their nearest register with zero friction. At halftime, the accountant aggregates balances across all registers (`sum()`). When writes dominate, LongAdder outpaces AtomicLong by orders of magnitude!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Striped64 Mimarisi ve LongAdder Derinliği: 1) Çekişme Azaltma (Contention Mitigation): `AtomicLong` tek bir bellek adresi üzerinde CAS yapar; thread sayısı arttıkça L1 cache geçersiz kılma (cache invalidation) fırtınası kopar. 2) `Cell[]` Dizisi: LongAdder, Doug Lea tarafından yazılan `Striped64` sınıfından türer. Çekişme tespit edildiğinde thread ID'sinin hash'ine göre dinamik olarak `Cell` dizisi büyütülür (2, 4, 8, 16...). Her `Cell` kendi içinde `\@Contended` anotasyonu ile 128 bayt doldurulmuştur (Padding), böylece False Sharing tamamen önlenir! 3) Ne Zaman AtomicLong, Ne Zaman LongAdder?: Eğer atomik bir sıra numarası (Sequence ID: `incrementAndGet()`) gerekiyorsa `AtomicLong` zorunludur. Eğer sadece metrik toplama, sayaç tutma veya istatistik yapılıyorsa kesinlikle `LongAdder` tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Internal Mechanics of Striped64: 1) The Contention Bottleneck of AtomicLong: AtomicLong forces all concurrent cores to serialize writes to a single memory offset, saturating the cache bus with MESI invalidation traffic. 2) Striped64 Cell Dynamism: LongAdder maintains a `base` variable. If a CAS on `base` fails due to contention, it allocates a `Cell[]` table sized to a power of two (capped at CPU core count). Each thread hashes into a specific `Cell`. Crucially, each `Cell` is annotated with `\@jdk.internal.vm.annotation.Contended`, enforcing 128-byte cache-line isolation to eliminate false sharing. 3) Architectural Trade-Off: `LongAdder.sum()` is an eventual snapshot, not an atomic instantaneous freeze. Use `AtomicLong` when sequential order or exact compare-and-set guards are required (`compareAndSet()`); use `LongAdder` for high-frequency telemetry, request counting, and metrics.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Yüksek Ölçekli Metrik Toplama için LongAdder
import java.util.concurrent.atomic.LongAdder;

public class MetricsCollector {
    // Saniyede 1 milyon HTTP isteğini kaydetmek için idealdir
    private final LongAdder requestCount = new LongAdder();

    public void onRequestReceived() {
        requestCount.increment(); // Çarpışmasız, dağıtık hücreye yazar
    }

    public long getTotalRequests() {
        return requestCount.sum(); // Tüm hücreleri toplar
    }

    void main() throws InterruptedException {
        var metrics = new MetricsCollector();
        int threads = 8;
        int reqPerThread = 500_000;

        Thread[] workers = new Thread[threads];
        for (int i = 0; i < threads; i++) {
            workers[i] = Thread.ofPlatform().start(() -> {
                for (int j = 0; j < reqPerThread; j++) {
                    metrics.onRequestReceived();
                }
            });
        }

        for (Thread t : workers) t.join();

        IO.println("Toplam Başarılı İstek: " + metrics.getTotalRequests()); // 4,000,000
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Banka hesap bakiyesi veya sıra numarası için LongAdder kullanabilir miyiz?' Yanıt: 'HAYIR! LongAdder atomik bir `compareAndSet` veya anlık kesin okuma sunmaz. `sum()` çağrısı yapılırken arka planda yazmalar devam edebilir. Sıra numarası için daima AtomicLong kullanılmalıdır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Critical Interview Distinction: 'Can we use LongAdder to issue unique order IDs or manage account balances?' Answer: 'NO! LongAdder does not provide an atomic `compareAndSet()` or transactional read-and-mutate guarantee. `sum()` yields an eventual aggregate, not an isolated lockstep snapshot. Unique sequences require AtomicLong.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LongAdder, yüksek çekişmeli yazma işlemlerinde AtomicLong'dan 10-100 kat daha hızlıdır.
- Striped64 hücreleri \@Contended ile pad edildiği için False Sharing yaşanmaz.
- Benzersiz sıra numarası (Sequence ID) için AtomicLong; metrikler için LongAdder seçilir.

🎯 Key Takeaways for the Interview

- LongAdder achieves 10-100x throughput over AtomicLong under high write contention.
- Striped64 cells are padded to prevent False Sharing across CPU cache lines.
- Use AtomicLong for atomic sequential IDs; use LongAdder for high-volume metrics.